



Outputs in Terraform

Introduction

Imagine you are using Terraform to deploy cloud resources.

- How do you get the public IP of an EC2 instance after deployment?
- How do you retrieve database connection details automatically?
- How can Terraform pass information between modules?

Terraform Outputs solve these problems!

Section 1: Learn

What is an Output in Terraform?

An output in Terraform is a way to retrieve and display information about resources created by Terraform.

Outputs help to:

1. Get useful information after infrastructure deployment.
2. Pass values between different Terraform modules.
3. Display key data (like instance IPs, database endpoints) after `terraform apply`.

Why Do We Need Outputs?

Without Outputs	With Outputs
Manually check cloud console for resource details	Automatically displays useful information
Difficult to pass values between modules	Outputs simplify data sharing



Without Outputs	With Outputs
Prone to human errors in fetching details	Ensures consistency and automation

Key Features of Terraform Outputs

1. **Extract information** like instance IP, database URL, or security group ID.
2. **Pass data between Terraform modules.**
3. **Use outputs in automation scripts or CI/CD pipelines.**

An Interesting Anecdote

Before Terraform outputs, cloud engineers had to **log in to AWS/GCP/Azure consoles** to find server details. With Terraform outputs, **this information is instantly available after deployment!**

Section 2: Practice

Step-by-Step Guide to Using Terraform Outputs

1. Defining an Output in Terraform

Example: Retrieving the **public IP of an EC2 instance**.

Modify **main.tf**:

```
provider "aws" {  
  region = "us-east-1"  
}  
  
resource "aws_instance" "web_server" {  
  ami      = "ami-12345678"  
  instance_type = "t2.micro"
```



```
}  
  
output "instance_ip" {  
  description = "The public IP of the EC2 instance"  
  value       = aws_instance.web_server.public_ip  
}
```

Run Terraform:

```
terraform apply
```

After successful deployment, Terraform will display:

Outputs:

```
instance_ip = "18.235.12.45"
```

Why is this useful?

- **Retrieves important details automatically after deployment.**
-

2. Using Outputs to Pass Data Between Modules

Step 1: Define Output in a Module

In `modules/network/main.tf`:

```
resource "aws_vpc" "my_vpc" {  
  cidr_block = "10.0.0.0/16"  
}  
  
output "vpc_id" {
```



```
value = aws_vpc.my_vpc.id
}
```

Step 2: Retrieve Output in Another Module

In **main.tf**:

```
module "network" {
  source = "../modules/network"
}

resource "aws_instance" "web_server" {
  ami          = "ami-12345678"
  instance_type = "t2.micro"
  vpc_security_group_ids = [module.network.vpc_id]
}
```

Run Terraform:

```
terraform apply
```

What happens here?

- The VPC ID from the network module is passed to the EC2 instance.
-

3. Formatting Outputs with Pretty Printing

You can format output values using Terraform's built-in functions.

```
output "formatted_message" {
  value = "Your EC2 instance is running with public IP:
${aws_instance.web_server.public_ip}"
}
```



```
}
```

After running Terraform:

Outputs:

```
formatted_message = "Your EC2 instance is running with public IP:
18.235.12.45"
```

Why is this important?

- Makes output messages more user-friendly.
-

4. Storing Output Values in a File

To store outputs in a file for later use:

```
terraform output -json > outputs.json
```

Why is this useful?

- Helps in **automation scripts** and **CI/CD pipelines**.
-

5. Using Sensitive Outputs

For security reasons, sensitive outputs (like passwords) should be hidden.

```
output "database_password" {
  value    = aws_db_instance.example.password
  sensitive = true
}
```

Now, Terraform will **not display the value in logs**.

Why is this useful?



- Protects sensitive credentials from being exposed.
-

Real-World Example

A **DevOps team** needs to:

1. Retrieve public IPs of newly launched EC2 instances.
2. Pass the VPC ID from one module to another.
3. Securely hide sensitive data like database passwords.

What did they do?

1. Used **Terraform outputs** to retrieve instance details.
2. Passed **network configurations using module outputs**.
3. Set **sensitive outputs** to hide passwords securely.

Result? The team **automated infrastructure retrieval, reduced manual effort, and improved security!**

Section 3: Know More

Frequently Asked Questions

1. **What is an output in Terraform?**
 - Outputs **retrieve and display resource information** after deployment.
2. **How do I use Terraform outputs in another module?**
 - Define an **output in one module** and reference it in another module.
3. **How do I store Terraform outputs in a file?**
 - Run:



```
terraform output -json > outputs.json
```

4. **Can Terraform outputs display sensitive data securely?**

- Yes, use the **sensitive = true** flag to hide sensitive values.

5. **Where can I practice Terraform outputs?**

- Use **AWS Free Tier** and experiment with Terraform configurations.

Conclusion

Terraform **Outputs** simplify retrieving, sharing, and securing infrastructure details.

Start by **defining simple outputs**, passing data between modules, and using **sensitive outputs**, and soon you'll be **automating cloud infrastructure effortlessly!**